

Computer Vision:

Week-12 (20 April--24 April) Theory Lectures

Dr. Irfan Ali

Assistant Professor (AI, Multimedia Gaming & Cybersecurity

& Program Coordinator of Multimedia & Gaming Deptt

Review

- Data Augmentation, Loss function, Optimizers and its types, Transfer learning, advanced models

- Object Detection — Principles, Architectures, and Metrics.

Object Detection

- **Object detection** is a task in computer vision where a model:
Finds objects in an image
(localization using bounding boxes)
Identifies what they are
(classification)

- **Key Components**
- **Bounding Box (Localization)**
 - Defined by coordinates: (x, y, width, height)
- **Class Label (Classification)**
 - Assigns category like “car”, “dog”, etc.
- **Confidence Score**
 - Probability that detection is correct

The Geometry of a Bounding Box

The smallest rectangle that contains all pixels of an object.

Representations:

Corner – based : $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$

Centroid – based : $[x_c, y_c, w, h]$

Coordinates are usually scaled between 0 and 1 relative to image width/height for model stability through Normalization.

- To calculate the boundary coordinates of a bounding box based on its center point and dimensions, use the following formulas:

From Corner to Centroid

$$w = x_{max} - x_{min}$$

$$h = y_{max} - y_{min}$$

$$x_c = x_{min} + \frac{w}{2}$$

$$y_c = y_{min} + \frac{h}{2}$$

From Centroid to Corner

$$x_{min} = x_c - \frac{w}{2} \quad y_{min} = y_c - \frac{h}{2}$$

$$x_{max} = x_c + \frac{w}{2} \quad y_{max} = y_c + \frac{h}{2}$$

Scenario: An algorithm detects a car with the following corner coordinates:

Top – left : $(50, 80)$

Bottom – right : $(150, 200)$

Result : *The centroid – based box is*
 $(100, 140, 100, 120)$.

Scenario: A YOLO-style model predicts a bounding box centered at (300, 400) with a width of 60 and a height of 80.

- *Find the pixel coordinates for the top-left and bottom-right corners.*
- *Result: The corner-based box is (270, 360, 330, 440).*

Challenges in Detection

- **Variable Outputs:** Unlike classification (fixed output size), an image might have 0 or 100 objects.
- **Multiple Scales:** Objects can be tiny (distant bird) or huge (close-up car).
- **Aspect Ratios:** Tall objects (pedestrians) vs. wide objects (limousines).
- **Occlusion:** Objects hiding behind other objects.

Challenges in Detection

Feature	Image Classification	Object Detection / Segmenting
Output	Assign one label to the	Identify and locate every instance.
Goal	whole image.	
Data	A fixed-length array (e.g.,	<i>A list of detections</i>
Structure	length 1000).	$[Class, X, Y, W, H]$.
Flexibility	Rigid: Cannot handle multiple objects.	Fluid: Grows or shrinks based on the scene.
Example	"This is a photo of a forest."	"There are 47 trees and 1 deer in this photo."

Challenges in Detection

The "Magnifying Glass" Problem

- Imagine looking at a high-resolution photo of a park.
- To see a **ladybug** on a leaf, you need to zoom in and look at a very small area.
- To see the **whole tree**, you need to step back and look at the entire image.
- If a computer uses a fixed-size "window" to look for objects, it will either be too big to see the ladybug or too small to realize it's looking at a tree.

Challenges in Detection

The Solution: The Image Pyramid

-The most common way to fix this is by creating an **Image Pyramid**. We take the original photo and create multiple versions of it at different sizes.

- **Bottom of the Pyramid:** The original, high-resolution image. Great for finding **tiny objects** (like a distant bird).
- **Top of the Pyramid:** A tiny, blurry version of the image. Great for finding **huge objects** (like a close-up car) because the computer can see the "whole shape" at once without getting lost in the details.

Challenges in Detection

Occlusion: The model plays a digital game of "connect the dots," using visible fragments and logic to identify an object even when it is partially hidden behind something else.

The Detection Pipeline Flowchart

- **Backbone:** Extracts features (e.g., ResNet).
- **Neck:** Aggregates features from different layers (e.g., FPN, PAN).
- **Head:** Predicts the classes and boxes.

Evolutionary Path: Two-Stage vs. One-Stage

- **Two-Stage (Region-based):** "Look, then Think." Focuses on high-quality proposals.
- **One-Stage (Regression-based):** "Look once." Focuses on speed and global context.
- **The Trade-off:** Accuracy (Two-Stage) vs. Efficiency (One-Stage).

Two-Stage Detectors:

The original **R-CNN** (Regions with CNN features) was like a very thorough, but very slow.

- **Stage 1:** It looks at an image and cuts out about 2,000 "interesting" looking patches (proposals).
- **Stage 2:** It runs each of those 2,000 patches through a deep neural network one by one to see what they are.
- **The Catch:** This was incredibly slow. It could take nearly a minute to process a single photo!

Faster R-CNN

Faster R-CNN (Region-based Convolutional Neural Network) is a landmark deep learning model designed for **object detection**. It was a major breakthrough because it was the first "near real-time" object detector to replace slow, traditional computer vision algorithms with a neural network that could "propose" where objects are

Faster R-CNN

In earlier versions (R-CNN and Fast R-CNN), the model relied on an external algorithm called **Selective Search** to find potential objects in an image. This was a "bottleneck" because it was slow and ran on the CPU.

How it Works

Faster R-CNN is known as a two-stage detector:

- Stage 1: Region Proposal

The image is passed through a base network (like ResNet or VGG) to create a feature map. The RPN then slides over this map and uses Anchor Boxes (pre-defined boxes of different sizes and ratios) to suggest where objects might be.

- Stage 2: Classification and Refinement

The proposed regions (Rols) are cropped from the feature map. These are then sent to a final layer that classifies what the object is (e.g., "dog," "car") and fine-tunes the coordinates of the bounding box to fit the object perfectly.

How it Works

While even faster "one-stage" models like **YOLO** (You Only Look Once) are now popular for mobile or high-speed applications, Faster R-CNN is still widely used in research and industry when **high accuracy** is more important than raw speed. It also serves as the foundation for **Mask R-CNN**, which is used for even more complex tasks like identifying the exact pixel-by-pixel outline of objects

One-Stage Detectors: The YOLO Revolution

-YOLO (You Only Look Once): Proposed by Joseph Redmon (2015).

- Concept: Divide the image into an $S \times S$ grid.
- The Grid Rule: If an object's center falls in a cell, that cell is responsible for detecting it.
- Parallelism: Prediction happens for all cells simultaneously.